

1. What Is Software Architecture?

We are called to be architects of the future, not its victims.

—R. Buckminster Fuller

Writing (on our part) and reading (on your part) a book about software architecture, which distills the experience of many people, presupposes that

1. having a reasonable software architecture is important to the successful development of a software system and
2. there is a sufficient body of knowledge about software architecture to fill up a book.

There was a time when both of these assumptions needed justification. Early editions of this book tried to convince readers that both of these assumptions are true and, once you were convinced, supply you with basic knowledge so that you could apply the practice of architecture yourself. Today, there seems to be little controversy about either aim, and so this book is more about the supplying than the convincing.

The basic principle of software architecture is every software system is constructed to satisfy an organization's business goals, and that the architecture of a system is a bridge between those (often abstract) business goals and the final (concrete) resulting system. While the path from abstract goals to concrete systems can be complex, the good news is that software architectures can be designed, analyzed, and documented using known techniques that will support the achievement of these business goals. The complexity can be tamed, made tractable.

These, then, are the topics for this book: the design, analysis, and documentation of architectures. We will also examine the influences, principally in the form of business goals that lead to quality attribute requirements, that inform these activities.

In this chapter, we will focus on architecture strictly from a software *engineering* point of view. That is, we will explore the value that a software architecture brings to a development project. Later chapters will take business and organizational perspectives.

1.1 What Software Architecture Is and What It Isn't

There are many definitions of software architecture, easily discoverable with a web search, but the one we like is this:

The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.

This definition stands in contrast to other definitions that talk about the system's "early" or "major" or "important" decisions. While it is true that many architectural decisions are made early, not all are—especially in Agile and spiral-development projects. It's also true that many decisions that are made early are not what we would consider architectural. Also, it's hard to look at a decision and tell whether it's "major." Sometimes only time will tell. And since deciding on an architecture is one of the architect's most important obligations, we need to know which decisions an architecture comprises.

Structures, by contrast, are fairly easy to identify in software, and they form a powerful tool for system design and analysis.

So, there we are: Architecture is about reasoning-enabling structures.

Let's look at some of the implications of our definition.

Architecture Is a Set of Software Structures

This is the first and most obvious implication of our definition. A structure is simply a set of elements held together by a relation. Software systems are composed of many structures, and no single structure can lay claim to being *the* architecture. Structures can be grouped into categories, and the categories themselves provide useful ways to think about the architecture. Architectural structures can be organized into three useful categories, which will play an important role in the design, documentation, and analysis of architectures:

1. Component-and-connector structures
2. Module structures
3. Allocation structures

We'll delve more into these types of structures in the next section.

Although software comprises an endless supply of structures, not all of them are architectural. For example, the set of lines of source code that contain the letter "z," ordered by increasing length from shortest to longest, is a software structure. But it's not a very interesting one, nor is it architectural. A structure is architectural if it supports reasoning about the system and the system's properties. The reasoning should be about an attribute of the system that is important to some stakeholder(s). These include properties such as the functionality achieved by the system, the system's ability to keep operating usefully in the face of faults or attempts to take it down, the ease or difficulty of making specific changes to the system, the system's responsiveness to user

requests, and many others. We will spend a great deal of time in this book exploring the relationship between architecture and *quality attributes* like these.

Thus, the set of architectural structures is neither fixed nor limited. What is architectural depends on what is useful to reason about in your context for your system.

Architecture Is an Abstraction

Since architecture consists of structures, and structures consist of elements¹ and relations, it follows that an architecture comprises software elements and how those elements relate to each other. This means that architecture specifically and intentionally omits certain information about elements that is not useful for reasoning about the system. Thus, an architecture is foremost an *abstraction* of a system that selects certain details and suppresses others. In all modern systems, elements interact with each other by means of interfaces that partition details about an element into public and private parts. Architecture is concerned with the public side of this division; private details of elements—details having to do solely with internal implementation—are not architectural. This abstraction is essential to taming the complexity of an architecture: We simply cannot, and do not want to, deal with all of the complexity all of the time. We want—and need—the understanding of a system’s architecture to be many orders of magnitude easier than understanding every detail about that system. You can’t keep every detail of a system of even modest size in your head; the point of architecture is to make it so you don’t have to.

¹ In this book we use the term “element” when we mean either a module or a component, and don’t want to distinguish between the two.

Architecture versus Design

Architecture is design, but not all design is architecture. That is, many design decisions are left unbound by the architecture—it is, after all, an abstraction—and depend on the discretion and good judgment of downstream designers and even implementers.

Every Software System Has a Software Architecture

Every system has an architecture, because every system has elements and relations. However, it does not follow that the architecture is known to anyone. Perhaps all of the people who designed the system are long gone, the documentation has vanished (or was never produced), the source code has been lost (or was never delivered), and all we have at hand is the executing binary code. This reveals the difference between the architecture of a system and the

representation of that architecture. Given that an architecture can exist independently of its description or specification, this raises the importance of *architecture documentation*, which is described in [Chapter 22](#).

Not All Architectures Are Good Architectures

Our definition is indifferent as to whether the architecture for a system is a good one or a bad one. An architecture may either support or hinder achieving the important requirements for a system. Assuming that we do not accept trial and error as the best way to choose an architecture for a system—that is, picking an architecture at random, building the system from it, and then hacking away and hoping for the best—this raises the importance of *architecture design*, which is treated in [Chapter 20](#) and *architecture evaluation*, which will be dealt with in [Chapter 21](#).

Architecture Includes Behavior

The behavior of each element is part of the architecture insofar as that behavior can help you reason about the system. The behavior of elements embodies how they interact with each other and with the environment. This is clearly part of our definition of architecture and will have an effect on the properties exhibited by the system, such as its runtime performance.

Some aspects of behavior are below the architect's level of concern. Nevertheless, to the extent that an element's behavior influences the acceptability of the system as a whole, this behavior must be considered part of the system's architectural design, and should be documented as such.

System and Enterprise Architectures

Two disciplines related to software architecture are system architecture and enterprise architecture. Both of these disciplines have broader concerns than software and affect software architecture through the establishment of constraints within which a software system, and its architect, must live.

System Architecture

A system's architecture is a representation of a system in which there is a mapping of functionality onto hardware and software components, a mapping of the software architecture onto the hardware architecture, and a concern for the human interaction with these components. That is, system architecture is concerned with the totality of hardware, software, and humans.

A system architecture will influence, for example, the functionality that is assigned to different processors and the types of networks that connect those

processors. The software architecture will determine how this functionality is structured and how the software programs residing on the various processors interact.

A description of the software architecture, as it is mapped to hardware and networking components, allows reasoning about qualities such as performance and reliability. A description of the system architecture will allow reasoning about additional qualities such as power consumption, weight, and physical dimensions.

When designing a particular system, there is frequently negotiation between the system architect and the software architect over the distribution of functionality and, consequently, the constraints placed on the software architecture.

Enterprise Architecture

Enterprise architecture is a description of the structure and behavior of an organization's processes, information flow, personnel, and organizational subunits. An enterprise architecture need not include computerized information systems—clearly, organizations had architectures that fit the preceding definition prior to the advent of computers—but these days enterprise architectures for all but the smallest businesses are unthinkable without information system support. Thus, a modern enterprise architecture is concerned with how software systems support the enterprise's business processes and goals. Typically included in this set of concerns is a process for deciding which systems with which functionality the enterprise should support.

An enterprise architecture will specify, for example, the data model that various systems use to interact. It will also specify rules for how the enterprise's systems interact with external systems.

Software is only one concern of enterprise architecture. How the software is used by humans to perform business processes and the standards that determine the computational environment are two other common concerns addressed by enterprise architecture.

Sometimes the software infrastructure that supports communication among systems and with the external world is considered a portion of the enterprise architecture; at other times this infrastructure is considered one of the systems within an enterprise. (In either case, the architecture of that infrastructure is a *software* architecture!) These two views will result in different management structures and spheres of influence for the individuals concerned with the infrastructure.

Are These Disciplines in Scope for This Book? Yes! (Well, No.)

The system and the enterprise provide environments for, and constraints on, the software architecture. The software architecture must live within the system and the enterprise, and increasingly is the focus for achieving the organization's business goals. Enterprise and system architectures share a great deal with software architectures. All can be designed, evaluated, and documented; all answer to requirements; all are intended to satisfy stakeholders; all consist of structures, which in turn consist of elements and relationships; all have a repertoire of patterns at their respective architects' disposal; and the list goes on. So to the extent that these architectures share commonalities with software architecture, they are in the scope of this book. But like all technical disciplines, each has its own specialized vocabulary and techniques, and we won't cover those. Copious other sources exist that do.

1.2 Architectural Structures and Views

Because architectural structures are at the heart of our definition and treatment of software architecture, this section will explore these concepts in more depth. These concepts are dealt with in much greater depth in [Chapter 22](#), where we discuss architecture documentation.

Architectural structures have counterparts in nature. For example, the neurologist, the orthopedist, the hematologist, and the dermatologist all have different views of the various structures of a human body, as illustrated in [Figure 1.1](#). Ophthalmologists, cardiologists, and podiatrists concentrate on specific subsystems. Kinesiologists and psychiatrists are concerned with different aspects of the entire arrangement's behavior. Although these views are pictured differently and have very different properties, all are inherently related and interconnected: Together they describe the architecture of the human body.

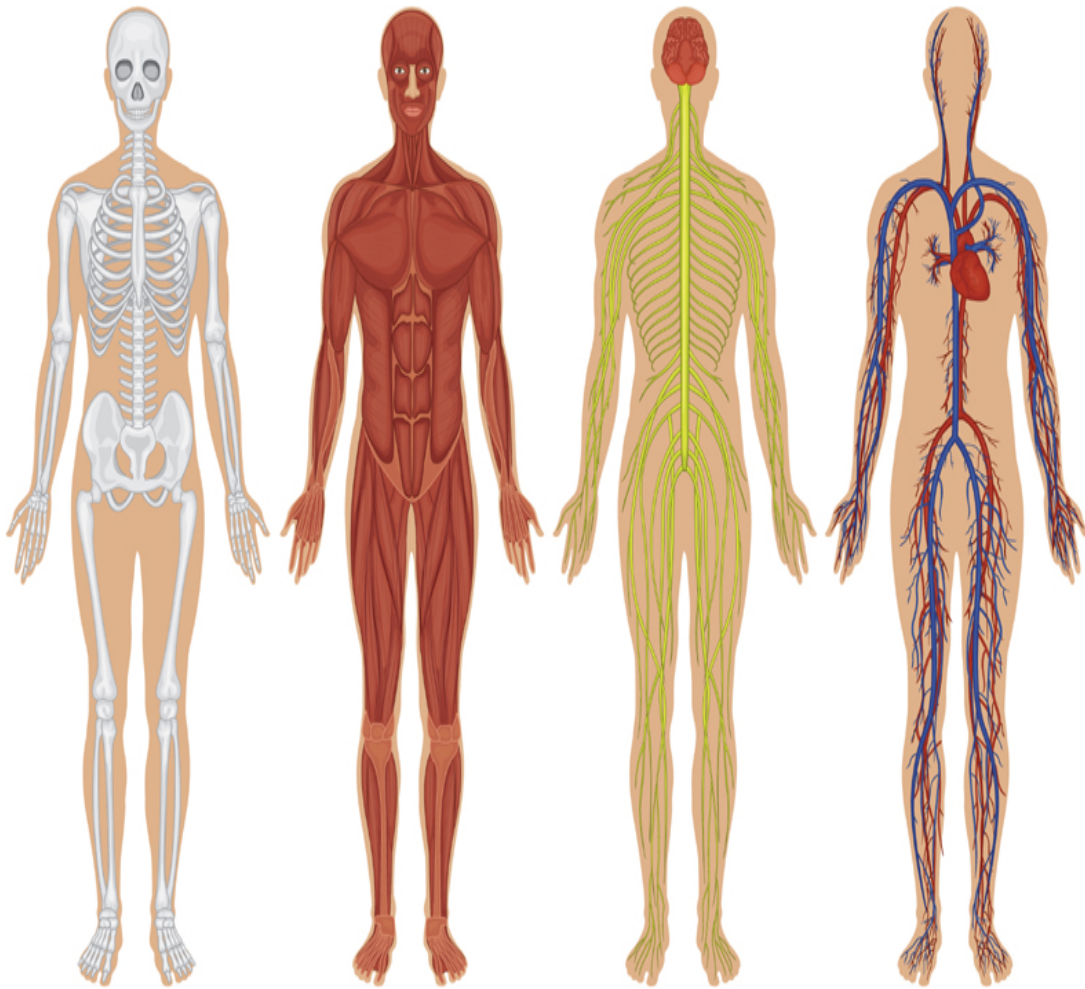


Figure 1.1 *Physiological structures.* (adapted from <https://courses.lumenlearning.com/ap1/chapter/structural-organization-of-the-human-body/>)

Architectural structures also have counterparts in human endeavors. For example, electricians, plumbers, heating and air conditioning specialists, roofers, and framers are each concerned with different structures in a building. You can readily see the qualities that are the focus of each of these structures.

So it is with software.

Three Kinds of Structures

Architectural structures can be divided into three major categories, depending on the broad nature of the elements they show and the kinds of reasoning they support:

1. *Component-and-connector (C&C) structures* focus on the way the elements interact with each other at runtime to carry out the system's functions. They describe how the system is structured as a set of elements that have runtime

behavior (components) and interactions (connectors). Components are the principal units of computation and could be services, peers, clients, servers, filters, or many other types of runtime element. Connectors are the communication vehicles among components, such as call-return, process synchronization operators, pipes, or others. C&C structures help answer questions such as the following:

- What are the major executing components and how do they interact at runtime?
- What are the major shared data stores?
- Which parts of the system are replicated?
- How does data progress through the system?
- Which parts of the system can run in parallel?
- Can the system's structure change as it executes and, if so, how?

By extension, these structures are crucially important for asking questions about the system's runtime properties, such as performance, security, availability, and more.

C&C structures are the most common ones that we see, but two other categories of structures are important and should not be overlooked.

[Figure 1.2](#) shows a sketch of a C&C structure of a system using an informal notation that is explained in the figure's key. The system contains a shared repository that is accessed by servers and an administrative component. A set of client tellers can interact with the account servers and communicate among themselves using a publish-subscribe connector.

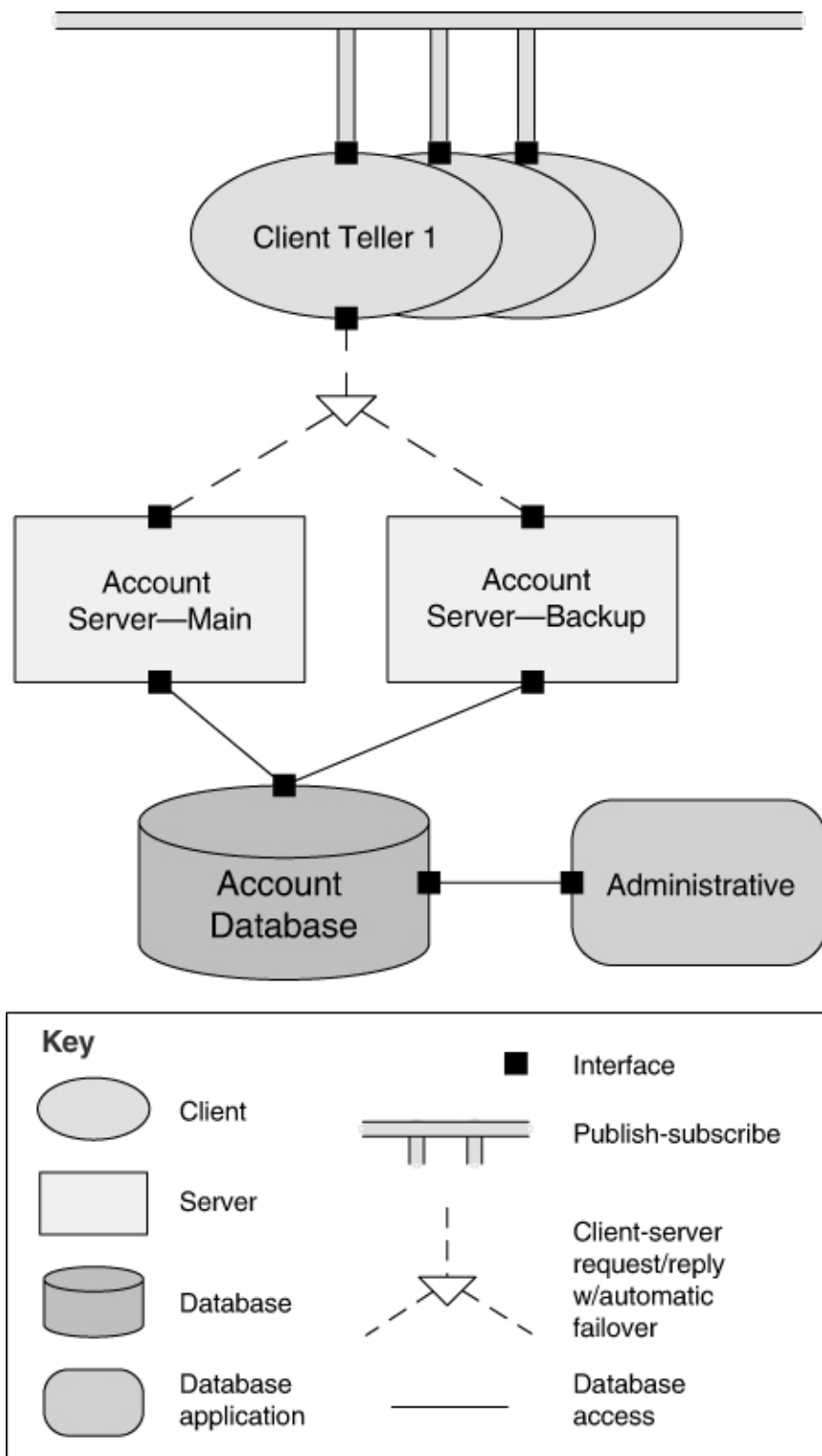


Figure 1.2 A component-and-connector structure

2. *Module structures* partition systems into implementation units, which in this book we call *modules*. Module structures show how a system is structured as a set of code or data units that have to be constructed or procured. Modules are

assigned specific computational responsibilities, and are the basis of work assignments for programming teams. In any module structure, the elements are modules of some kind (perhaps classes, packages, layers, or merely divisions of functionality, all of which are units of implementation). Modules represent a static way of considering the system. Modules are assigned areas of functional responsibility; there is less emphasis in these structures on how the resulting software manifests itself at runtime. Module implementations include packages, classes, and layers. Relations among modules in a module structure include uses, generalization (or “is-a”), and “is part of.” [Figures 1.2](#) and [1.3](#) show examples of module elements and relations, respectively, using the Unified Modeling Language (UML) notation.

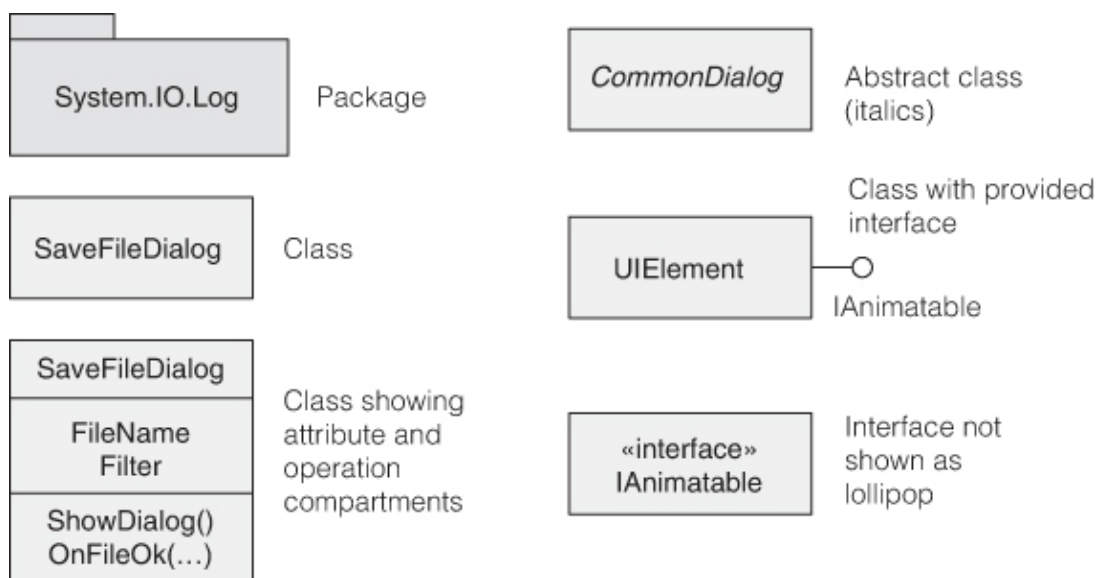


Figure 1.3 *Module elements in UML*

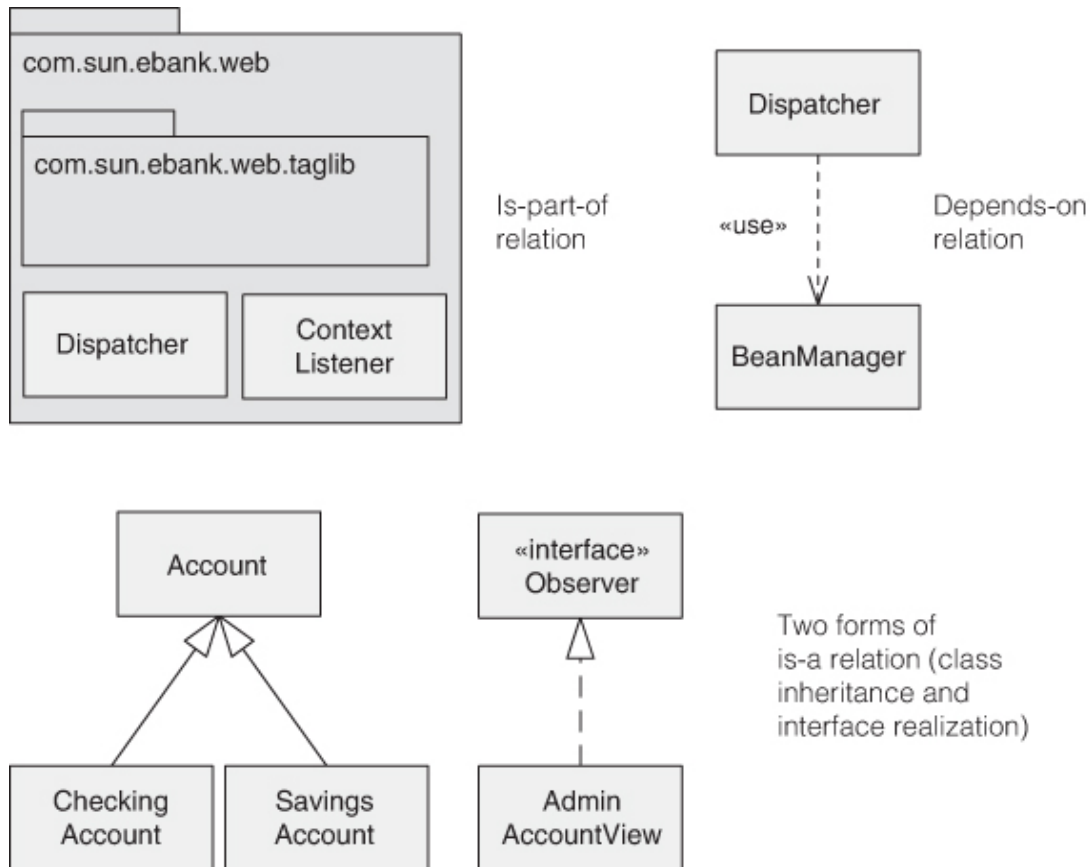


Figure 1.4 *Module relations in UML*

Module structures allow us to answer questions such as the following:

- What is the primary functional responsibility assigned to each module?
- What other software elements is a module allowed to use?
- What other software does it actually use and depend on?
- What modules are related to other modules by generalization or specialization (i.e., inheritance) relationships?

Module structures convey this information directly, but they can also be used to answer questions about the impact on the system when the responsibilities assigned to each module change. Thus, module structures are the primary tools for reasoning about a system's modifiability.

3. *Allocation structures* establish the mapping from software structures to the system's nonsoftware structures, such as its organization, or its development, test, and execution environments. Allocation structures answer questions such as the following:

- Which processor(s) does each software element execute on?

- In which directories or files is each element stored during development, testing, and system building?
- What is the assignment of each software element to development teams?

Some Useful Module Structures

Useful module structures include:

- *Decomposition structure.* The units are modules that are related to each other by the “is-a-submodule-of” relation, showing how modules are decomposed into smaller modules recursively until the modules are small enough to be easily understood. Modules in this structure represent a common starting point for design, as the architect enumerates what the units of software will have to do and assigns each item to a module for subsequent (more detailed) design and eventual implementation. Modules often have products (such as interface specifications, code, and test plans) associated with them. The decomposition structure determines, to a large degree, the system’s modifiability. That is, do changes fall within the purview of a few (preferably small) modules? This structure is often used as the basis for the development project’s organization, including the structure of the documentation, and the project’s integration and test plans. [Figure 1.5](#) shows an example of a decomposition structure.

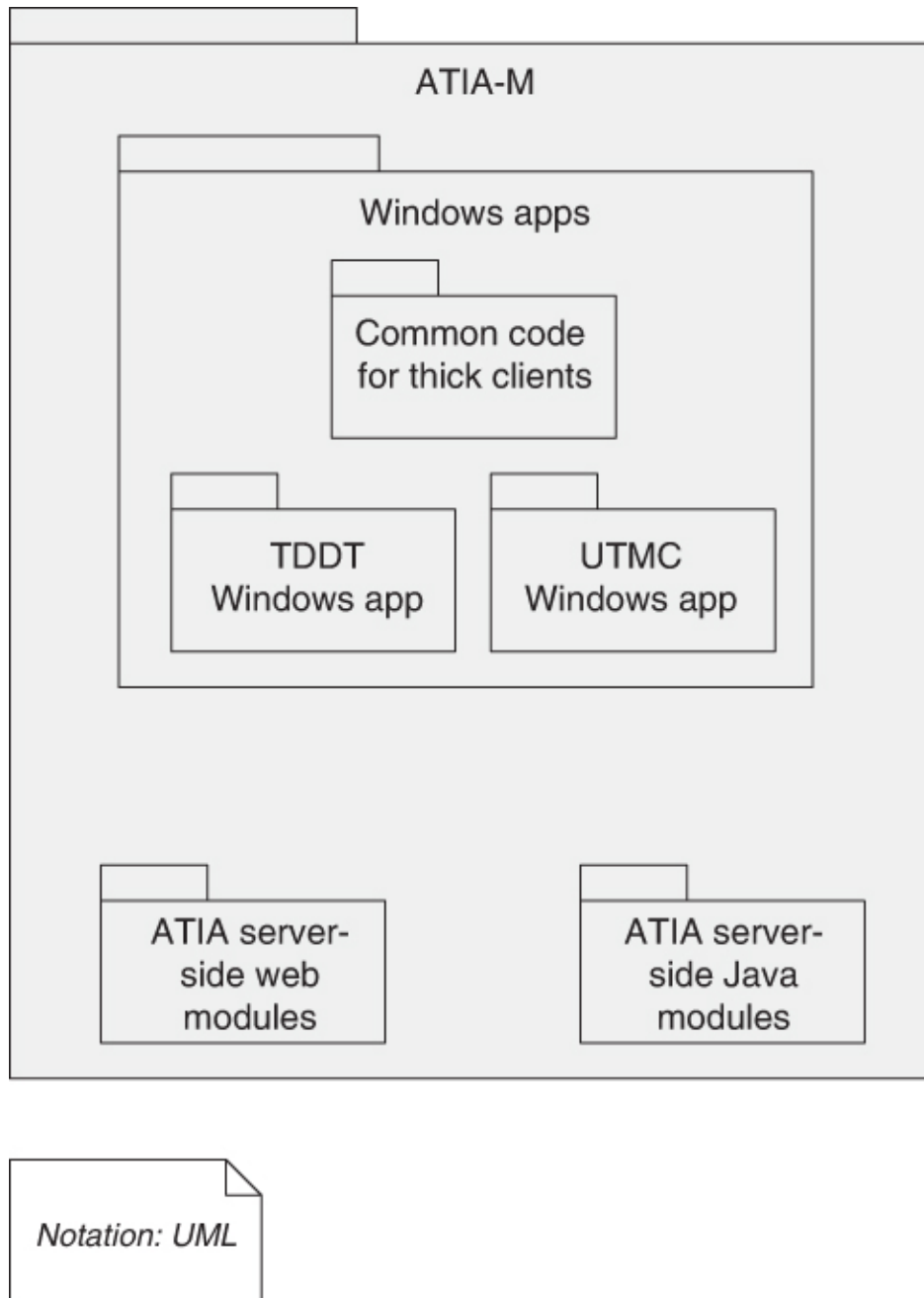


Figure 1.5 A decomposition structure

- *Uses structure.* In this important but often overlooked structure, the units are also modules, and perhaps classes. The units are related by the *uses* relation, a specialized form of dependency. One unit of software uses another if the correctness of the first requires the presence of a correctly functioning version (as opposed to a stub) of the second. The uses structure is used to engineer systems that can be extended to add functionality, or from which useful functional subsets can be extracted. The ability to easily create a subset of a system allows for incremental development. This structure is also the

basis for measuring social debt—the amount of communication that actually is, as opposed to merely should be, taking place among teams—as it defines which teams should be talking to each other. Figure 1.6 shows a uses structure and highlights the modules that must be present in an increment if the module admin.client is present.

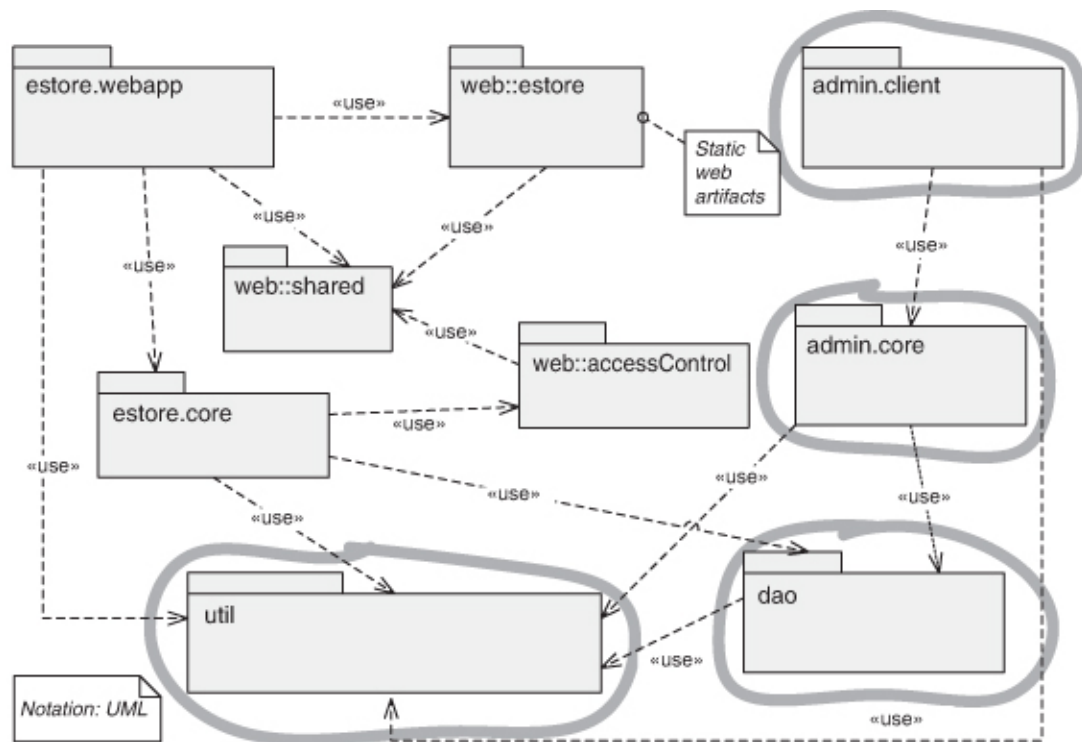


Figure 1.6 Uses structure

- *Layer structure.* The modules in this structure are called layers. A layer is an abstract “virtual machine” that provides a cohesive set of services through a managed interface. Layers are allowed to use other layers in a managed fashion; in strictly layered systems, a layer is only allowed to use a single other layer. This structure imbues a system with portability—that is, the ability to change the underlying virtual machine. Figure 1.7 shows a layer structure of the UNIX System V operating system.

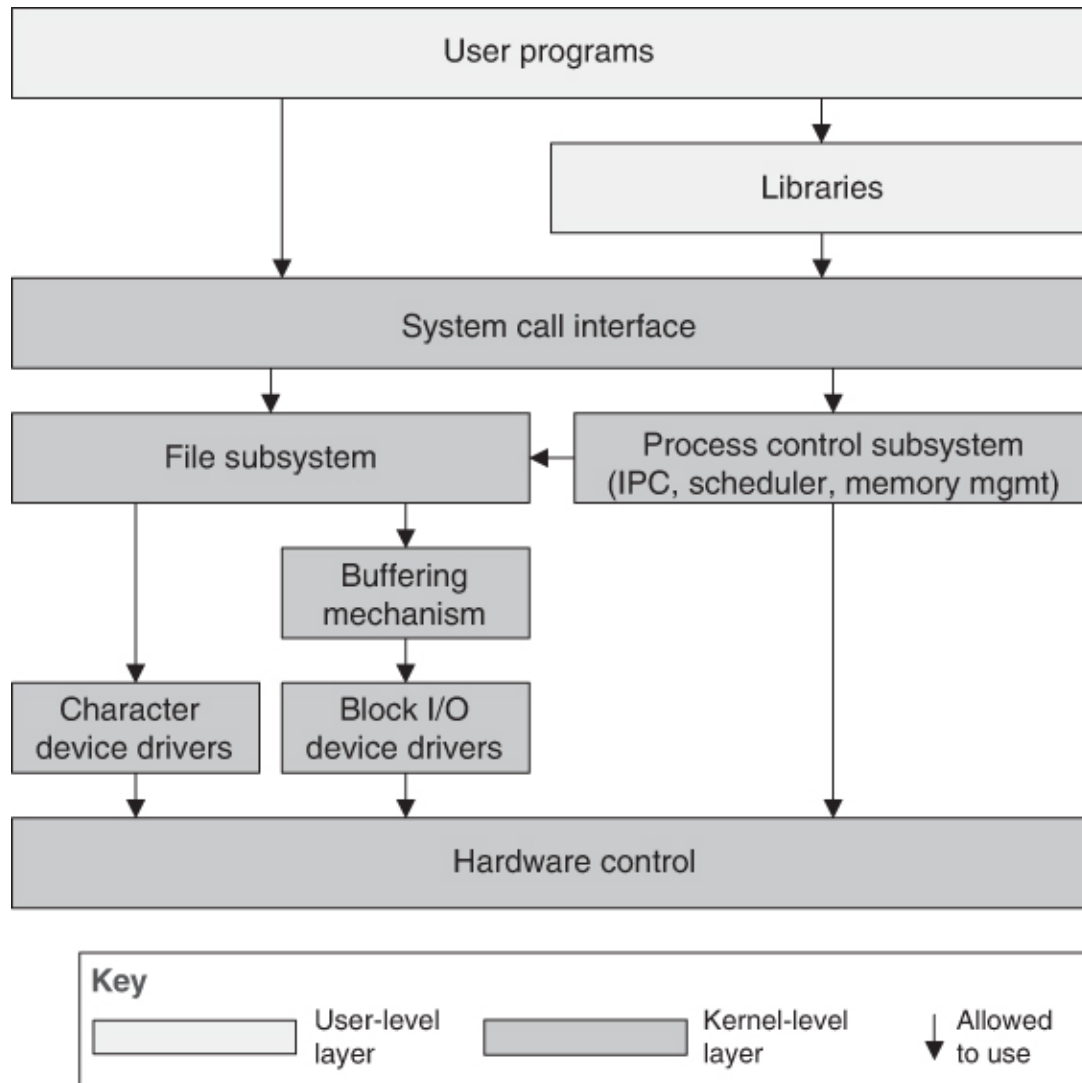


Figure 1.7 *Layer structure*

- *Class (or generalization) structure.* The modules in this structure are called classes, and they are related through an “inherits-from” or “is-an-instance-of” relation. This view supports reasoning about collections of similar behavior or capability and parameterized differences. The class structure allows one to reason about reuse and the incremental addition of functionality. If any documentation exists for a project that has followed an object-oriented analysis and design process, it is typically this structure. [Figure 1.8](#) shows a generalization structure taken from an architectural expert tool.

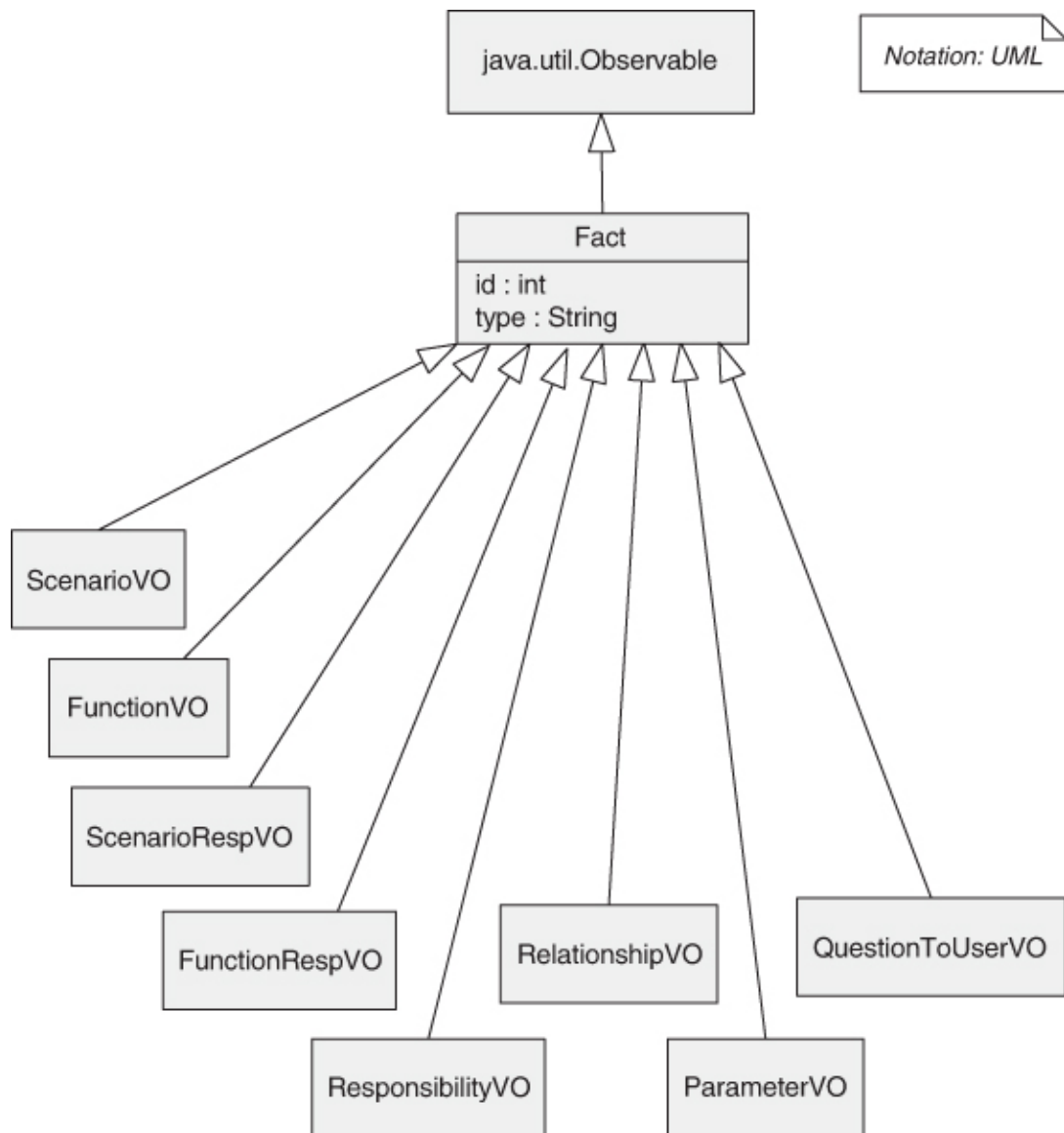


Figure 1.8 Generalization structure

- *Data model.* The data model describes the static information structure in terms of data entities and their relationships. For example, in a banking system, entities will typically include Account, Customer, and Loan. Account has several attributes, such as account number, type (savings or checking), status, and current balance. A relationship may dictate that one customer can have one or more accounts, and one account is associated with one or more customers. [Figure 1.9](#) shows an example of a data model.

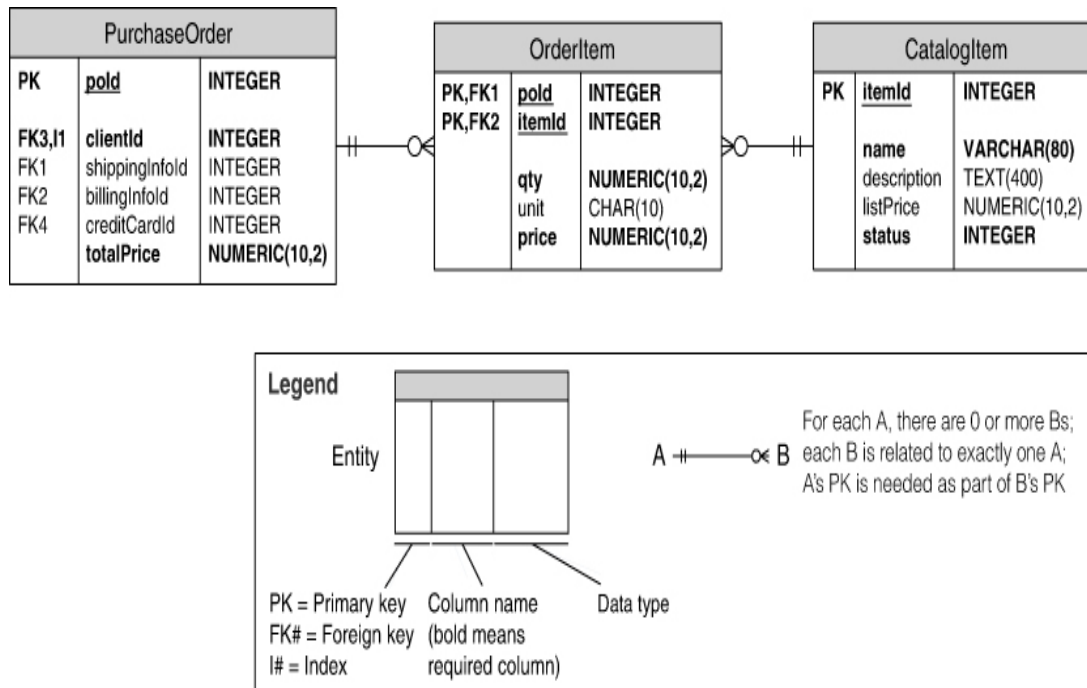


Figure 1.9 Data model

Some Useful C&C Structures

C&C structures show a runtime view of the system. In these structures, the modules just described have all been compiled into executable forms. Thus, all C&C structures are orthogonal to the module-based structures and deal with the dynamic aspects of a running system. For example, one code unit (module) could be compiled into a single service that is replicated thousands of times in an execution environment. Or 1,000 modules can be compiled and linked together to produce a single runtime executable (component).

The relation in all C&C structures is *attachment*, showing how the components and the connectors are hooked together. (The connectors themselves can be familiar constructs such as “invokes.”) Useful C&C structures include:

- *Service structure.* The units here are services that interoperate through a service coordination mechanism, such as messages. The service structure is an important structure to help engineer a system composed of components that may have been developed independently of each other.
- *Concurrency structure.* This C&C structure allows the architect to determine opportunities for parallelism and the locations where resource contention may occur. The units are components and the connectors are their communication mechanisms. The components are arranged into “logical threads.” A logical thread is a sequence of computations that could be allocated to a separate physical thread later in the design process. The

concurrency structure is used early in the design process to identify and manage issues associated with concurrent execution.

Some Useful Allocation Structures

Allocation structures define how the elements from C&C or module structures map onto things that are not software—typically hardware (possibly virtualized), teams, and file systems. Useful allocation structures include:

- *Deployment structure.* The deployment structure shows how software is assigned to hardware processing and communication elements. The elements are software elements (usually a process from a C&C structure), hardware entities (processors), and communication pathways. Relations are “allocated-to,” showing on which physical units the software elements reside, and “migrates-to” if the allocation is dynamic. This structure can be used to reason about performance, data integrity, security, and availability. It is of particular interest in distributed systems, and is the key structure involved in the achievement of the quality attribute of deployability (see [Chapter 5](#)). [Figure 1.10](#) shows a simple deployment structure in UML.

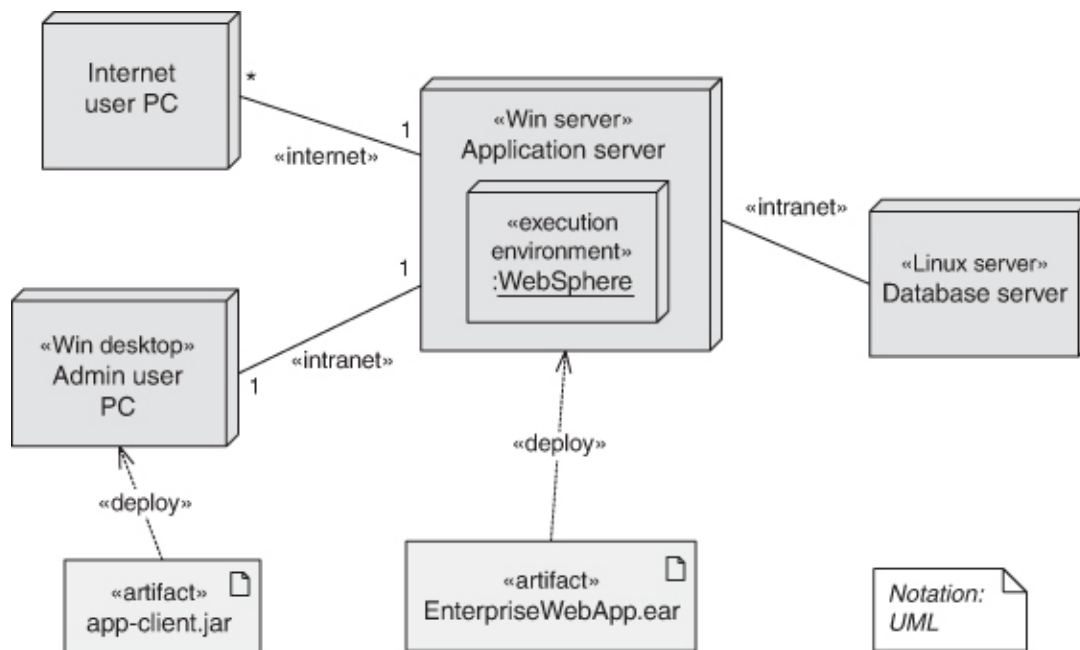


Figure 1.10 *Deployment structure*

- *Implementation structure.* This structure shows how software elements (usually modules) are mapped to the file structures in the system’s development, integration, test, or configuration control environments. This is critical for the management of development activities and build processes.

- *Work assignment structure.* This structure assigns responsibility for implementing and integrating the modules to the teams who will carry out these tasks. Having a work assignment structure be part of the architecture makes it clear that the decision about who does the work has architectural as well as management implications. The architect will know the expertise required on each team. Amazon's decision to devote a single team to each of its microservices, for example, is a statement about its work assignment structure. On large development projects, it is useful to identify units of functional commonality and assign those to a single team, rather than having them be implemented by everyone who needs them. This structure will also determine the major communication pathways among the teams: regular web conferences, wikis, email lists, and so forth.

[Table 1.1](#) summarizes these structures. It lists the meaning of the elements and relations in each structure and tells what each might be used for.

Table 1.1 *Useful Architectural Structures*

	Software Structure	Element Types	Relations	Useful for	Quality Concerns Affected
Module structures	Decomposition	Module	Is a submodule of	Resource allocation and project structuring and planning; encapsulation	Modifiability
	Uses	Module	Uses (i.e., requires the correct presence of)	Designing subsets and extensions	“Subsetability,” extensibility
	Layers	Layer	Allowed to use the services of; provides abstraction to	Incremental development; implementing systems on top of “virtual machines”	Portability, modifiability
	Class	Class, object	Is an instance of; is a generalization of	In object-oriented systems, factoring out commonality; planning extensions of functionality	Modifiability, extensibility
	Data model	Data entity	{one, many}-to-{one, many}; generalizes; specializes	Engineering global data structures for consistency and performance	Modifiability, performance
C&C structures	Service	Service, service registry	Attachment (via message-passing)	Scheduling analysis; performance analysis; robustness analysis	Interoperability, availability, modifiability
	Concurrency	Processes, threads	Attachment (via communication and synchronization mechanisms)	Identifying locations where resource contention exists, opportunities for parallelism	Performance
Allocation structures	Deployment	Components, hardware elements	Allocated to; migrates to	Mapping software elements to system elements	Performance, security, energy, availability, deniability

	Implementa tion	Modules, file structure	Stored in	Configuration control, integration, test activities	Development efficiency
	Work assignment	Modules, organizatio nal units	Assigned to	Project management, best use of expertise and available resources, management of commonality	Development efficiency

Relating Structures to Each Other

Each of these structures provides a different perspective and design handle on a system, and each is valid and useful in its own right. Although the structures give different system perspectives, they are not independent. Elements of one structure will be related to elements of other structures, and we need to reason about these relations. For example, a module in a decomposition structure may be manifested as one, part of one, or several components in one of the C&C structures, reflecting its runtime alter-ego. In general, mappings between structures are many to many.

Figure 1.11 shows a simple example of how two structures might relate to each other. The image on the left shows a module decomposition view of a tiny client-server system. In this system, two modules must be implemented: the client software and the server software. The image on the right shows a C&C view of the same system. At runtime, ten clients are running and accessing the server. Thus, this little system has two modules and 11 components (and ten connectors).

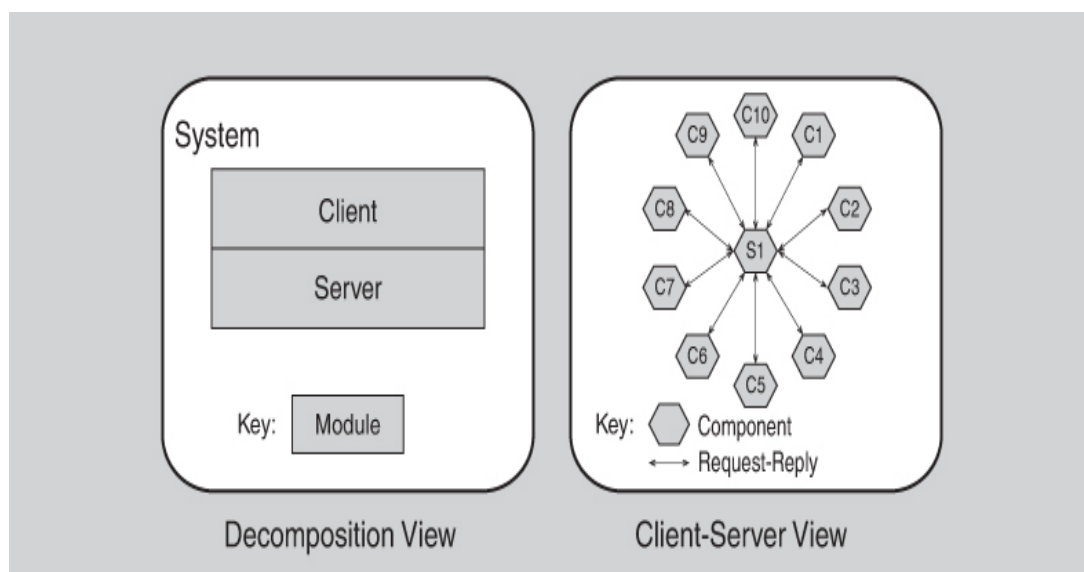


Figure 1.11 *Two views of a client-server system*

Whereas the correspondence between the elements in the decomposition structure and the client-server structure is obvious, these two views are used for very different things. For example, the view on the right could be used for performance analysis, bottleneck prediction, and network traffic management, which would be extremely difficult or impossible to do with the view on the left. (In [Chapter 9](#), we'll learn about the map-reduce pattern, in which copies of simple, identical functionality are distributed across hundreds or thousands of processing nodes—one module for the whole system, but one component per node.)

Individual projects sometimes consider one structure to be dominant and cast other structures, when possible, in terms of the dominant structure. Often, the dominant structure is the module decomposition structure, and for good reason: It tends to spawn the project structure, since it mirrors the team structure of development. In other projects, the dominant structure might be a C&C structure that shows how the system's functionality and/or critical quality attributes are achieved at runtime.

Fewer Is Better

Not all systems warrant consideration of many architectural structures. The larger the system, the more dramatic the difference between these structures tends to be; but for small systems, we can often get by with fewer structures. For example, instead of working with each of several C&C structures, usually a single one will do. If there is only one process, then the process structure collapses to a single node and need not be explicitly represented in the design. If no distribution will occur (that is, if the system is implemented on a single processor), then the deployment structure is trivial and need not be considered further. In general, you should design and document a structure only if doing so brings a positive return on the investment, usually in terms of decreased development or maintenance costs.

Which Structures to Choose?

We have briefly described a number of useful architectural structures, and many more are certainly possible. Which ones should an architect choose to work on? Which ones should the architect choose to document? Surely not all of them. A good answer is that you should think about how the various structures available to you provide insight and leverage into the system's most important quality attributes, and then choose the ones that will play the best role in delivering those attributes.

Architectural Patterns

In some cases, architectural elements are composed in ways that solve particular problems. These compositions have been found to be useful over time and over many different domains, so they have been documented and disseminated. These compositions of architectural elements, which provide packaged strategies for solving some of the problems facing a system, are called patterns. Architectural patterns are discussed in detail in Part 2 of this book.

1.3 What Makes a “Good” Architecture?

There is no such thing as an inherently good or bad architecture. Architectures are either more or less fit for some purpose. A three-tier layered service-oriented architecture may be just the ticket for a large enterprise’s web-based B2B system but completely wrong for an avionics application. An architecture carefully crafted to achieve high modifiability does not make sense for a throw-away prototype (and vice versa!). One of the messages of this book is that architectures can, in fact, be *evaluated*—one of the great benefits of paying attention to them—but such evaluation only makes sense in the context of specific stated goals.

Nevertheless, some rules of thumb should be followed when designing most architectures. Failure to apply any of these guidelines does not automatically mean that the architecture will be fatally flawed, but it should at least serve as a warning sign that should be investigated. These rules can be applied proactively for greenfield development, to help build the system “right.” Or they can be applied as analysis heuristics, to understand the potential problem areas in existing systems and to guide the direction of its evolution.

We divide our observations into two clusters: process recommendations and product (or structural) recommendations. Our process recommendations are as follows:

1. A software (or system) architecture should be the product of a single architect or a small group of architects with an identified technical leader. This approach is important to give the architecture its conceptual integrity and technical consistency. This recommendation holds for agile and open source projects as well as “traditional” ones. There should be a strong connection between the architects and the development team, to avoid “ivory tower,” impractical designs.
2. The architect (or architecture team) should, on an ongoing basis, base the architecture on a prioritized list of well-specified quality attribute requirements. These will inform the tradeoffs that always occur. Functionality matters less.

3. The architecture should be documented using *views*. (A view is simply a representation of one or more architectural structures.) The views should address the concerns of the most important stakeholders in support of the project timeline. This might mean minimal documentation at first, with the documentation then being elaborated later. Concerns usually are related to construction, analysis, and maintenance of the system, as well as education of new stakeholders.
4. The architecture should be evaluated for its ability to deliver the system's important quality attributes. This should occur early in the life cycle, when it returns the most benefit, and repeated as appropriate, to ensure that changes to the architecture (or the environment for which it is intended) have not rendered the design obsolete.
5. The architecture should lend itself to incremental implementation, to avoid having to integrate everything at once (which almost never works) as well as to discover problems early. One way to do this is via the creation of a "skeletal" system in which the communication paths are exercised but which at first has minimal functionality. This skeletal system can be used to "grow" the system incrementally, refactoring as necessary.

Our structural rules of thumb are as follows:

1. The architecture should feature well-defined modules whose functional responsibilities are assigned on the principles of information hiding and separation of concerns. The information-hiding modules should encapsulate things likely to change, thereby insulating the software from the effects of those changes. Each module should have a well-defined interface that encapsulates or "hides" the changeable aspects from other software that uses its facilities. These interfaces should allow their respective development teams to work largely independently of each other.
2. Unless your requirements are unprecedented—possible, but unlikely—your quality attributes should be achieved by using well-known architectural patterns and tactics (described in [Chapters 4 through 13](#)) specific to each attribute.
3. The architecture should never depend on a particular version of a commercial product or tool. If it must, it should be structured so that changing to a different version is straightforward and inexpensive.
4. Modules that produce data should be separate from modules that consume data. This tends to increase modifiability because changes are frequently confined to either the production or the consumption side of data. If new data is added, both sides will have to change, but the separation allows for a staged (incremental) upgrade.

5. Don't expect a one-to-one correspondence between modules and components. For example, in systems with concurrency, multiple instances of a component may be running in parallel, where each component is built from the same module. For systems with multiple threads of concurrency, each thread may use services from several components, each of which was built from a different module.
6. Every process should be written so that its assignment to a specific processor can be easily changed, perhaps even at runtime. This is a driving force in the increasing trends toward virtualization and cloud deployment, as we will discuss in [Chapters 16](#) and [17](#).
7. The architecture should feature a small number of simple component interaction patterns. That is, the system should do the same things in the same way throughout. This practice will aid in understandability, reduce development time, increase reliability, and enhance modifiability.
8. The architecture should contain a specific (and small) set of resource contention areas, whose resolution is clearly specified and maintained. For example, if network utilization is an area of concern, the architect should produce (and enforce) for each development team guidelines that will result in acceptable levels of network traffic. If performance is a concern, the architect should produce (and enforce) time budgets.

1.4 Summary

The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.

There are three categories of structures:

- Module structures show the system as a set of code or data units that have to be constructed or procured.
- Component-and-connector structures show the system as a set of elements that have runtime behavior (components) and interactions (connectors).
- Allocation structures show how elements from module and C&C structures relate to nonsoftware structures (such as CPUs, file systems, networks, and development teams).

Structures represent the primary engineering leverage points of an architecture. Each structure brings with it the power to manipulate one or more quality attributes. Collectively, structures represent a powerful approach for creating the architecture (and later, for analyzing it and explaining it to its stakeholders). And, as we will see in [Chapter 22](#), the structures that the architect has chosen as

engineering leverage points are also the primary candidates to choose as the basis for architecture documentation.

Every system has a software architecture, but this architecture may or may not be documented and disseminated.

There is no such thing as an inherently good or bad architecture. Architectures are either more or less fit for some purpose.

1.5 For Further Reading

If you're keenly interested in software architecture as a field of study, you might be interested in reading some of the pioneering work. Most of it does not mention "software architecture" at all, as this phrase evolved only in the mid-1990s, so you'll have to read between the lines.

Edsger Dijkstra's 1968 paper on the T.H.E. operating system introduced the concept of layers [[Dijkstra 68](#)]. The early work of David Parnas laid many conceptual foundations, including information hiding [[Parnas 72](#)], program families [[Parnas 76](#)], the structures inherent in software systems [[Parnas 74](#)], and the uses structure to build subsets and supersets of systems [[Parnas 79](#)]. All of Parnas's papers can be found in the more easily accessible collection of his important papers [[Hoffman 00](#)]. Modern distributed systems owe their existence to the concept of cooperating sequential processes that (among others) Sir C. A. R. (Tony) Hoare was instrumental in conceptualizing and defining [[Hoare 85](#)].

In 1972, Dijkstra and Hoare, along with Ole-Johan Dahl, argued that programs should be decomposed into independent components with small and simple interfaces. They called their approach structured programming, but arguably this was the debut of software architecture [[Dijkstra 72](#)].

Mary Shaw and David Garlan, together and separately, produced a major body of work that helped create the field of study we call software architecture. They established some of its fundamental principles and, among other things, catalogued a seminal family of architectural styles (a concept similar to patterns), several of which appear in this chapter as architectural structures. Start with [[Garlan 95](#)].

Software architectural patterns have been extensively catalogued in the series *Pattern-Oriented Software Architecture* [[Buschmann 96](#) and others]. We also deal with architectural patterns in Part 2 of this book.

Early papers on architectural views as used in industrial development projects are [[Soni 95](#)] and [[Kruchten 95](#)]. The former grew into a book [[Hofmeister 00](#)] that presents a comprehensive picture of using views in development and analysis.

A number of books have focused on practical implementation issues associated with architectures, such as George Fairbanks' *Just Enough Software Architecture* [Fairbanks 10], Woods and Rozanski's *Software Systems Architecture* [Woods 11], and Martin's *Clean Architecture: A Craftsman's Guide to Software Structure and Design* [Martin 17].

1.6 Discussion Questions

1. Is there a different definition of software architecture that you are familiar with? If so, compare and contrast it with the definition given in this chapter. Many definitions include considerations like “rationale” (stating the reasons why the architecture is what it is) or how the architecture will evolve over time. Do you agree or disagree that these considerations should be part of the definition of software architecture?
2. Discuss how an architecture serves as a basis for analysis. What about decision making? What kinds of decision making does an architecture empower?
3. What is architecture's role in project risk reduction?
4. Find a commonly accepted definition of *system architecture* and discuss what it has in common with software architecture. Do the same for *enterprise architecture*.
5. Find a published example of a software architecture. Which structures are shown? Given its purpose, which structures *should* have been shown? What analysis does the architecture support? Critique it: What questions do you have that the representation does not answer?
6. Sailing ships have architectures, which means they have “structures” that lend themselves to reasoning about the ship's performance and other quality attributes. Look up the technical definitions for barque, brig, cutter, frigate, ketch, schooner, and sloop. Propose a useful set of “structures” for distinguishing and reasoning about ship architectures.
7. Aircraft have architectures that can be characterized by how they resolve some major design questions, such as engine location, wing location, landing gear layout, and more. For many decades, most jet aircraft designed for passenger transport have the following characteristics:
 - Engines housed in nacelles slung underneath the wing (as opposed to engines built into the wings, or engines mounted on the rear of the fuselage)
 - Wings that join the fuselage at the bottom (as opposed to the top or middle)

First, do an online search to find an example and a counter-example of this type of design from each of the following manufacturers: Boeing, Embraer, Tupolev, Bombardier. Next, do some online research and answer the following question: What qualities important to aircraft does this design provide?

2. Why Is Software Architecture Important?

Ah, to build, to build!

That is the noblest art of all the arts.

—Henry Wadsworth Longfellow

If architecture is the answer, what was the question?

This chapter focuses on why architecture matters from a technical perspective. We will examine a baker's dozen of the most important reasons. You can use these reasons to motivate the creation of a new architecture, or the analysis and evolution of an existing system's architecture.

1. An architecture can either inhibit or enable a system's driving quality attributes.
2. The decisions made in an architecture allow you to reason about and manage change as the system evolves.
3. The analysis of an architecture enables early prediction of a system's qualities.
4. A documented architecture enhances communication among stakeholders.
5. The architecture is a carrier of the earliest, and hence most-fundamental, hardest-to-change design decisions.
6. An architecture defines a set of constraints on subsequent implementation.
7. The architecture dictates the structure of an organization, or vice versa.
8. An architecture can provide the basis for incremental development.
9. An architecture is the key artifact that allows the architect and the project manager to reason about cost and schedule.
10. An architecture can be created as a transferable, reusable model that forms the heart of a product line.
11. Architecture-based development focuses attention on the assembly of components, rather than simply on their creation.
12. By restricting design alternatives, architecture channels the creativity of developers, reducing design and system complexity.
13. An architecture can be the foundation for training of a new team member.

Even if you already believe us that architecture is important and don't need that point hammered home 13 more times, think of these 13 points (which form the outline for this chapter) as 13 useful ways to use architecture in a project, or to justify the resources devoted to architecture.

2.1 Inhibiting or Enabling a System's Quality Attributes

A system's ability to meet its desired (or required) quality attributes is substantially determined by its architecture. If you remember nothing else from this book, remember that.

This relationship is so important that we've devoted all of Part 2 of this book to expounding that message in detail. Until then, keep these examples in mind as a starting point:

- If your system requires high performance, then you need to pay attention to managing the time-based behavior of elements, their use of shared resources, and the frequency and volume of their interelement communication.
- If modifiability is important, then you need to pay attention to assigning responsibilities to elements and limiting the interactions (coupling) of those elements so that the majority of changes to the system will affect a small number of those elements. Ideally, each change will affect just a single element.
- If your system must be highly secure, then you need to manage and protect interelement communication and control which elements are allowed to access which information. You may also need to introduce specialized elements (such as an authorization mechanism) into the architecture to set up a strong "perimeter" to guard against intrusion.
- If you want your system to be safe and secure, you need to design in safeguards and recovery mechanisms.
- If you believe that scalability of performance will be important to the success of your system, then you need to localize the use of resources to facilitate introduction of higher-capacity replacements, and you must avoid hard-coding in resource assumptions or limits.
- If your projects need the ability to deliver incremental subsets of the system, then you must manage intercomponent usage.
- If you want the elements from your system to be reusable in other systems, then you need to restrict interelement coupling so that when you extract an element, it does not come out with too many attachments to its current environment to be useful.

The strategies for these and other quality attributes are supremely architectural. But an architecture alone cannot guarantee the functionality or quality required of a system. Poor downstream design or implementation decisions can always undermine an adequate architectural design. As we like to say (*mostly* in jest): What the architecture giveth, the implementation may taketh away. Decisions at

all stages of the life cycle—from architectural design to coding and implementation and testing—affect system quality. Therefore, quality is not completely a function of an architectural design. But that’s where it starts.

2.2 Reasoning about and Managing Change

This is a corollary to the previous point.

Modifiability—the ease with which changes can be made to a system—is a quality attribute (and hence covered by the arguments in the previous section), but it is such an important quality that we have awarded it its own spot in the List of Thirteen. The software development community is coming to grips with the fact that roughly 80 percent of a typical software system’s total cost occurs *after* initial deployment. Most systems that people work on are in this phase. Many programmers and software designers *never* get to work on new development—they work under the constraints of the existing architecture and the existing body of code. Virtually all software systems change over their lifetimes, to accommodate new features, to adapt to new environments, to fix bugs, and so forth. But the reality is that these changes are often fraught with difficulty.

Every architecture, no matter what it is, partitions possible changes into three categories: local, nonlocal, and architectural.

- A local change can be accomplished by modifying a single element—for example, adding a new business rule to a pricing logic module.
- A nonlocal change requires multiple element modifications but leaves the underlying architectural approach intact—for example, adding a new business rule to a pricing logic module, then adding new fields to the database that this new business rule requires, and then revealing the results of applying the rule in the user interface.
- An architectural change affects the fundamental ways in which the elements interact with each other and will probably require changes all over the system—for example, changing a system from single-threaded to multi-threaded.

Obviously, local changes are the most desirable, so an *effective* architecture is one in which the most common changes are local, and hence easy to make. Nonlocal changes are not as desirable but do have the virtue that they can usually be staged—that is, rolled out—in an orderly manner over time. For example, you might first make changes to add a new pricing rule, then make the changes to actually deploy the new rule.

Deciding when changes are essential, determining which change paths have the least risk, assessing the consequences of proposed changes, and arbitrating sequences and priorities for requested changes all require broad insight into the relationships, performance, and behaviors of system software elements. These

tasks are all part of the job description for an architect. Reasoning about the architecture and analyzing the architecture can provide the insights necessary to make decisions about anticipated changes. If you do not take this step, and if you do not pay attention to maintaining the conceptual integrity of your architecture, then you will almost certainly accumulate *architecture debt*. We deal with this subject in [Chapter 23](#).

2.3 Predicting System Qualities

This point follows from the previous two: Architecture not only imbues systems with qualities, but does so in a predictable way.

This may seem obvious, but it need not be the case. Then designing an architecture would consist of making a series of pretty much random design decisions, building the system, testing for quality attributes, and hoping for the best. Oops—not fast enough or hopelessly vulnerable to attacks? Start hacking.

Fortunately, it *is* possible to make quality predictions about a system based solely on an evaluation of its architecture. If we know that certain kinds of architectural decisions lead to certain quality attributes in a system, then we can make those decisions and rightly expect to be rewarded with the associated quality attributes. After the fact, when we examine an architecture, we can determine whether those decisions have been made, and confidently predict that the architecture will exhibit the associated qualities.

This point and the previous point, taken together, mean that architecture largely determines system qualities and—even better!—we know how it does so, and we know how to make it do so.

Even if you don't perform the quantitative analytic modeling sometimes necessary to ensure that an architecture will deliver its prescribed benefits, this principle of evaluating decisions based on their quality attribute implications is invaluable for at least spotting potential trouble early.

2.4 Communication among Stakeholders

One point made in [Chapter 1](#) is that an architecture is an abstraction, and that is useful because it represents a simplified model of the whole system that (unlike the infinite details of the whole system) you can keep in your head. So can others on your team. Architecture represents a common abstraction of a system that most, if not all, of the system's stakeholders can use as a basis for creating mutual understanding, negotiating, forming consensus, and communicating with each other. The architecture—or at least parts of it—are sufficiently abstract that most nontechnical people can understand it to the extent they need to, particularly with some coaching from the architect, and yet that abstraction can be refined

into sufficiently rich technical specifications to guide implementation, integration, testing, and deployment.

Each stakeholder of a software system—customer, user, project manager, coder, tester, and so on—is concerned with different characteristics of the system that are affected by its architecture. For example:

- the user is concerned that the system is fast, reliable, and available when needed;
- the customer (who pays for the system) is concerned that the architecture can be implemented on schedule and according to budget;
- the manager is worried that (in addition to cost and schedule concerns) the architecture will allow teams to work largely independently, interacting in disciplined and controlled ways; and
- the architect is worried about strategies to achieve all of those goals.

Architecture provides a common language in which different concerns can be expressed, negotiated, and resolved at a level that is intellectually manageable even for large, complex systems. Without such a language, it is difficult to understand large systems sufficiently to make the early decisions that influence both quality and usefulness. Architectural analysis, as we will see in [Chapter 21](#), both depends on this level of communication and enhances it.

[Chapter 22](#), on architecture documentation, covers stakeholders and their concerns in greater depth.

***“What Happens When I Push This Button?”: Architecture as a Vehicle for
Stakeholder Communication***

The project review droned on and on. The government-sponsored development was behind schedule and over budget, and it was large enough that these lapses were attracting the U.S. Congress’s attention. And now the government was making up for past neglect by holding a marathon come-one-come-all review session. The contractor had recently undergone a buyout, which hadn’t helped matters. It was the afternoon of the second day, and the agenda called for presentation of the software architecture. The young architect—an apprentice to the chief architect for the system—was bravely explaining how the software architecture for the massive system would enable it to meet its very demanding real-time, distributed, high-reliability requirements. He had a solid presentation and a solid architecture to present. It was sound and sensible. But the audience—about 30 government representatives who had varying roles in the management and oversight of this sticky project—was tired. Some of them were even thinking that perhaps they should have gone into real estate instead

of enduring another one of these marathon let's-finally-get-it-right-this-time reviews.

The slide showed, in semiformal box-and-line notation, what the major software elements were in a runtime view of the system. The names were all acronyms, suggesting no semantic meaning without explanation, which the young architect gave. The lines showed data flow, message passing, and process synchronization. The elements were internally redundant, as the architect was explaining. "In the event of a failure," he began, using a laser pointer to denote one of the lines, "a restart mechanism triggers along this path when. . ."

"What happens when the mode select button is pushed?" interrupted one of the audience members. He was a government attendee representing the user community for this system.

"Beg your pardon?" asked the architect.

"The mode select button," he said. "What happens when you push it?"

"Um, that triggers an event in the device driver, up here," began the architect, laser-pointing. "It then reads the register and interprets the event code. If it's mode select, well, then, it signals the blackboard, which in turn signals the objects that have subscribed to that event. . ."

"No, I mean what does the system *do*," interrupted the questioner. "Does it reset the displays? And what happens if this occurs during a system reconfiguration?"

The architect looked a little surprised and flicked off the laser pointer. This was not an architectural question, but since he was an architect and therefore fluent in the requirements, he knew the answer. "If the command line is in setup mode, the displays will reset," he said. "Otherwise, an error message will be put on the control console, but the signal will be ignored." He put the laser pointer back on. "Now, the restart mechanism that I was talking about. . ."

"Well, I was just wondering," said the users' delegate. "Because I see from your chart that the display console is sending signal traffic to the target location module."

"What *should* happen?" asked another member of the audience, addressing the first questioner. "Do you really want the user to get mode data during its reconfiguring?" And for the next 45 minutes, the architect watched as the audience consumed his time slot by debating what the correct behavior of the system was supposed to be in various esoteric states—an absolutely essential conversation that should have happened when the requirements were being formulated but, for whatever reason, had not.

The debate was not architectural, but the architecture (and the graphical rendition of it) had sparked debate. It is natural to think of architecture as the basis for communication among some of the stakeholders besides the architects and developers: Managers, for example, use the architecture to create teams and

allocate resources among them. But users? The architecture is invisible to users, after all; why should they latch on to it as a tool for system understanding?

The fact is that they do. In this case, the questioner had sat through two days of viewgraphs all about function, operation, user interface, and testing. But it was the first slide on architecture that—even though he was tired and wanted to go home—made him realize he didn’t understand something. Attendance at many architecture reviews has convinced me that seeing the system in a new way prods the mind and brings new questions to the surface. For users, architecture often serves as that new way, and the questions that a user poses will be behavioral in nature. In a memorable architecture evaluation exercise a few years ago, the user representatives were much more interested in what the system was going to do than in how it was going to do it, and naturally so. Up until that point, their only contact with the vendor had been through its marketers. The architect was the first legitimate expert on the system to whom they had access, and they didn’t hesitate to seize the moment.

Of course, careful and thorough requirements specifications would ameliorate this, but for a variety of reasons they are not always created or available. In their absence, a specification of the architecture often serves to trigger questions and improve clarity. It is probably more prudent to recognize this possibility than to resist it.

Sometimes such an exercise will reveal unreasonable requirements, whose utility can then be revisited. A review of this type that emphasizes synergy between requirements and architecture would have let the young architect in our story off the hook by giving him a place in the overall review session to address that kind of information. And the user representative wouldn’t have felt like a fish out of water, asking his question at a clearly inappropriate moment.

—PCC

2.5 Early Design Decisions

Software architecture is a manifestation of the earliest design decisions about a system, and these early bindings carry enormous weight with respect to the system’s remaining development, its deployment, and its maintenance life. It is also the earliest point at which these important design decisions affecting the system can be scrutinized.

Any design, in any discipline, can be viewed as a sequence of decisions. When painting a picture, an artist decides on the material for the canvas and the media for recording—oil paint, watercolor, crayon—even before the picture is begun. Once the picture is begun, other decisions are immediately made: Where is the first line, what is its thickness, what is its shape? All of these early design decisions have a strong influence on the final appearance of the picture, and each

decision constrains the many decisions that follow. Each decision, in isolation, might appear innocent enough, but the early ones in particular have disproportionate weight simply because they influence and constrain so much of what follows.

So it is with architecture design. An architecture design can also be viewed as a set of decisions. Changing these early decisions will cause a ripple effect, in terms of the additional decisions that must now be changed. Yes, sometimes the architecture must be refactored or redesigned, but this is not a task we undertake lightly—because the “ripple” might turn into an avalanche.

What are these early design decisions embodied by software architecture? Consider:

- Will the system run on one processor or be distributed across multiple processors?
- Will the software be layered? If so, how many layers will there be? What will each one do?
- Will components communicate synchronously or asynchronously? Will they interact by transferring control or data, or both?
- Will the information that flows through the system be encrypted?
- Which operating system will we use?
- Which communication protocol will we choose?

Imagine the nightmare of having to change any of these or a myriad of other related decisions. Decisions like these begin to flesh out some of the structures of the architecture and their interactions.

2.6 Constraints on Implementation

If you want your implementation to conform to an architecture, then it must conform to the design decisions prescribed by the architecture. It must have the set of elements prescribed by the architecture; these elements must interact with each other in the fashion prescribed by the architecture; and each element must fulfill its responsibility to the other elements as prescribed by the architecture. Each of these prescriptions is a constraint on the implementer.

Element builders must be fluent in the specifications of their individual elements but they may not be aware of the architectural tradeoffs—the architecture (or architect) simply constrains them in such a way as to meet the tradeoffs. A classic example is when an architect assigns performance budgets to the pieces of software involved in some larger piece of functionality. If each software unit stays within its budget, the overall transaction will meet its performance requirement.

Implementers of each of the constituent pieces may not know the overall budget, but only their own.

Conversely, the architects need not be experts in all aspects of algorithm design or the intricacies of the programming language—although they should certainly know enough not to design something that is difficult to build. Architects, however, are the people responsible for establishing, analyzing, and enforcing the architectural decisions and tradeoffs.

2.7 Influences on Organizational Structure

Not only does architecture prescribe the structure of the system being developed, but that structure becomes engraved in the structure of the development project (and sometimes the structure of the entire organization). The normal method for dividing up the labor in a large project is to assign different groups different portions of the system to construct. This so-called work-breakdown structure of a system is manifested in the architecture in the work assignment structure described in [Chapter 1](#). Because the architecture includes the broadest decomposition of the system, it is typically used as the basis for the work-breakdown structure. The work-breakdown structure in turn dictates units of planning, scheduling, and budget; interteam communication channels; configuration control and file-system organization; integration and test plans and procedures; and even project minutiae such as how the project intranet is organized and who sits with whom at the company picnic. Teams communicate with each other in terms of the interface specifications for their elements. The maintenance activity, when launched, will also reflect the software structure, with teams formed to maintain specific elements from the architecture—the database, the business rules, the user interface, the device drivers, and so forth.

A side effect of establishing the work-breakdown structure is to freeze some aspects of the software architecture. A group that is responsible for one of the subsystems may resist having its responsibilities distributed across other groups. If these responsibilities have been formalized in a contractual relationship, changing responsibilities could become expensive or even litigious.

Thus, once the architecture has been agreed upon, it becomes very costly—for managerial and business reasons—to significantly modify it. This is one argument (among many) for analyzing the software architecture for a large system before settling on a specific choice.

2.8 Enabling Incremental Development

Once an architecture has been defined, it can serve as the basis for incremental development. The first increment can be a skeletal system in which at least some

of the infrastructure—how the elements initialize, communicate, share data, access resources, report errors, log activity, and so forth—is present, but much of the system’s application functionality is not.

Building the infrastructure and building the application functionality can go hand in hand. Design and build a little infrastructure to support a little end-to-end functionality; repeat until done.

Many systems are built as skeletal systems that can be extended using plug-ins, packages, or extensions. Examples include the R language, Visual Studio Code, and most web browsers. The extensions, when added, provide additional functionality over and above what is present in the skeleton. This approach aids the development process by ensuring that the system is executable early in the product’s life cycle. The fidelity of the system increases as extensions are added, or early versions are replaced by more complete versions of these parts of the software. In some cases, the parts may be low-fidelity versions or prototypes of the final functionality; in other cases, they may be *surrogates* that consume and produce data at the appropriate rates but do little else. Among other things, this allows potential performance (and other) problems to be identified early in the product’s life cycle.

This practice gained attention in the early 2000s through the ideas of Alistair Cockburn and his notion of a “walking skeleton.” More recently, it has been adopted by those employing MVP (minimum viable product) as a strategy for risk reduction.

The benefits of incremental development include a reduction of the potential risk in the project. If the architecture is for a family of related systems, the infrastructure can be reused across the family, lowering the per-system cost of each.

2.9 Cost and Schedule Estimates

Cost and schedule estimates are an important tool for the project manager. They help the project manager acquire the necessary resources as well as monitor progress on the project. One of the duties of an architect is to help the project manager create cost and schedule estimates early in the project’s life cycle. While top-down estimates are useful for setting goals and apportioning budgets, cost estimations based on a bottom-up understanding of the system’s pieces are typically more accurate than those based purely on top-down system knowledge.

As we have said, the organizational and work-breakdown structure of a project is almost always based on its architecture. Each team or individual responsible for a work item will be able to make more accurate estimates for their piece than a project manager can, and will feel more ownership in making those estimates

come true. But the best cost and schedule estimates will typically emerge from a consensus between the top-down estimates (created by the architect and the project manager) and the bottom-up estimates (created by the developers). The discussion and negotiation that result from this process create a far more accurate estimate than use of either approach by itself.

It helps if the requirements for a system have been reviewed and validated. The more up-front knowledge you have about the scope, the more accurate the cost and schedule estimates will be.

[Chapter 24](#) delves into the use of architecture in project management.

2.10 Transferable, Reusable Model

The earlier in the life cycle reuse is applied, the greater the benefit that can be achieved from this practice. While code reuse offers a benefit, reuse of architectures provides opportunities for tremendous leverage for systems with similar requirements. When architectural decisions can be reused across multiple systems, all of the early-decision consequences we described in earlier sections are also transferred to those systems.

A product line or family is a set of systems that are all built using the same set of shared assets—software components, requirements documents, test cases, and so forth. Chief among these assets is the architecture that was designed to handle the needs of the entire family. Product-line architects choose an architecture (or a family of closely related architectures) that will serve all envisioned members of the product line. The architecture defines what is fixed for all members of the product line and what is variable.

Product lines represent a powerful approach to multi-system development that has shown order-of-magnitude payoffs in time to market, cost, productivity, and product quality. The power of architecture lies at the heart of this paradigm. Similar to other capital investments, architectures for product lines become a developing organization's shared asset.

2.11 Architecture Allows Incorporation of Independently Developed Elements

Whereas earlier software paradigms focused on *programming* as the prime activity, with progress measured in lines of code, architecture-based development often focuses on *composing* or *assembling elements* that are likely to have been developed separately, even independently, from each other. This composition is possible because the architecture defines the elements that can be incorporated into the system. The architecture constrains possible replacements (or additions) according to how they interact with their environment, how they receive and

relinquish control, which data they consume and produce, how they access data, and which protocols they use for communication and resource sharing. We elaborate on these ideas in [Chapter 15](#).

Commercial off-the-shelf components, open source software, publicly available apps, and networked services are all examples of independently developed elements. The complexity and ubiquity of integrating many independently developed elements into your system has spawned an entire industry of software tools, such as Apache Ant, Apache Maven, MSBuild, and Jenkins.

For software, the payoffs can take the following forms:

- Decreased time to market (it should be easier to use someone else's ready solution than to build your own)
- Increased reliability (widely used software should have its bugs ironed out already)
- Lower cost (the software supplier can amortize development cost across its customer base)
- Flexibility (if the element you want to buy is not terribly special-purpose, it's likely to be available from several sources, which in turn increases your buying leverage)

An *open system* is one that defines a set of standards for software elements—how they behave, how they interact with other elements, how they share data, and so forth. The goal of an open system is to enable, and even encourage, many different suppliers to be able to produce elements. This can avoid “vendor lock-in,” a situation in which a single vendor is the only one who can provide an element and charges a premium price for doing so. Open systems are enabled by an architecture that defines the elements and their interactions.

2.12 Restricting the Vocabulary of Design Alternatives

As useful architectural solutions are collected, it becomes clear that although software elements can be combined in more or less infinite ways, there is something to be gained by voluntarily restricting ourselves to a relatively small number of choices of elements and their interactions. By doing so, we minimize the design complexity of the system we are building.

A software engineer is not an *artiste* where creativity and freedom are paramount. Instead, engineering is about discipline, and discipline comes, in part, by *restricting* the vocabulary of alternatives to proven solutions. Examples of these proven design solutions include tactics and patterns, which will be discussed

extensively in Part 2. Reusing off-the-shelf elements is another approach to restricting your design vocabulary.

Restricting your design vocabulary to proven solutions can yield the following benefits:

- Enhanced reuse
- More regular and simpler designs that are more easily understood and communicated and bring more reliably predictable outcomes
- Easier analysis with greater confidence
- Shorter selection time
- Greater interoperability

Unprecedented designs are risky. Proven designs are, well, proven. This is not to say that software design can never be innovative or offer new and exciting solutions. It can. But these solutions should not be invented for the sake of novelty; rather, they should be sought when existing solutions are insufficient to solve the problem at hand.

Properties of software follow from the choice of architectural tactics or patterns. Tactics and patterns that are more desirable for a particular problem should improve the resulting design solution, perhaps by making it easier to arbitrate conflicting design constraints, by increasing insights into poorly understood design contexts, and by helping surface inconsistencies in requirements. We will discuss architectural tactics and patterns in Part 2.

2.13 A Basis for Training

The architecture, including a description of how the elements interact with each other to carry out the required behavior, can serve as the first introduction to the system for new project members. This reinforces our point that one important use of software architecture is to support and encourage communication among the various stakeholders. The architecture serves as a common reference point for all of these people.

Module views are excellent means of showing someone the structure of a project: who does what, which teams are assigned to which parts of the system, and so forth. Component-and-connector views are excellent choices for explaining how the system is expected to work and accomplish its job. Allocation views show a new project member where their assigned part fits into the project's development or deployment environment.

2.14 Summary

Software architecture is important for a wide variety of technical and nontechnical reasons. Our List of Thirteen includes the following benefits:

1. An architecture will inhibit or enable a system's driving quality attributes.
2. The decisions made in an architecture allow you to reason about and manage change as the system evolves.
3. The analysis of an architecture enables early prediction of a system's qualities.
4. A documented architecture enhances communication among stakeholders.
5. The architecture is a carrier of the earliest, and hence most-fundamental, hardest-to-change design decisions.
6. An architecture defines a set of constraints on subsequent implementation.
7. The architecture dictates the structure of an organization, or vice versa.
8. An architecture can provide the basis for incremental development.
9. An architecture is the key artifact that allows the architect and the project manager to reason about cost and schedule.
10. An architecture can be created as a transferable, reusable model that forms the heart of a product line.
11. Architecture-based development focuses attention on the assembly of components, rather than simply on their creation.
12. By restricting design alternatives, architecture productively channels the creativity of developers, reducing design and system complexity.
13. An architecture can be the foundation for training of a new team member.

2.15 For Further Reading

The Software Architect Elevator: Redefining the Architect's Role in the Digital Enterprise by Gregor Hohpe describes the unique ability of architects to interact with people at all levels inside and outside an organization, and facilitate stakeholder communication [[Hohpe 20](#)].

The granddaddy of papers about architecture and organization is by [[Conway 68](#)]. Conway's law states that "organizations which design systems . . . are constrained to produce designs which are copies of the communication structures of these organizations."

Cockburn's notion of the walking skeleton is described in *Agile Software Development: The Cooperative Game* [[Cockburn 06](#)].

A good example of an open systems architecture standard is AUTOSAR, developed for the automotive industry ([autosar.org](https://www.autosar.org)).

For a comprehensive treatment on building software product lines, see [[Clements 16](#)]. Feature-based product line engineering is a modern, automation-centered approach to building product lines that expands the scope from software to systems engineering. A good summary may be found at [[INCOSE 19](#)].

2.16 Discussion Questions

1. If you remember nothing else from this book, remember . . . what? Extra credit for not peeking.
2. For each of the 13 reasons why architecture is important articulated in this chapter, take the contrarian position: Propose a set of circumstances under which architecture is not necessary to achieve the result indicated. Justify your position. (Try to come up with different circumstances for each of the 13 reasons.)
3. This chapter argues that architecture brings a number of tangible benefits. How would you measure the benefits, on a particular project, of each of the 13 points?
4. Suppose you want to introduce architecture-centric practices to your organization. Your management is open to the idea, but wants to know the ROI for doing so. How would you respond?
5. Prioritize the list of 13 reasons in this chapter according to some criteria that are meaningful to you. Justify your answer. Or, if you could choose only two or three of the reasons to promote the use of architecture in a project, which would you choose and why?